The only thing that survives the process

Animated explainer · src/memory.js

Messages die with the run. memory.md does not. It is a plain markdown file beside the code, read at the start of the next run and searchable on demand. No database, no cache, no index.

9 scenes · 1 markdown file · 2 access paths · 4 tags

CONTENTS

What is in this explainer

01 · Why this file exists 3

02 · The format 4

03 · Parsing 5

04 · Two access paths 6

05 · Writing 7

06 · The digest 8

07 · Searching 9

08 · The honest limit 10

09 · Recap 11

Page numbers refer to the folio at the foot of each page.

01 · Why this file exists

Everything else dies with the process.

SRC/MEMORY.JS · *source*

```
// messages = working memory inside one run
// memory.md = the only thing that survives
export const MEMORY_FILE = 'src/memory.md';
export const MEMORY_TAGS = ['fact', 'pref', 'fix', 'gotcha'];
```

Within a run — messages carry the conversation.

Across runs — memory.md carries lessons.

For humans — open it, diff it, delete a wrong line.

> **remember** A memory module with a cache is a memory module that lies. memory.md is the store.
> **On screen:** Everything inside a run lives in messages. When the process exits, it is gone. memory.md is the one artifact that survives, and it is deliberately human-readable. There is no database and no cache. Every method reads the file, so hand edits cannot drift.

02 · The format

One bullet. One memory.

SRC/MEMORY.MD · *text*

```
# memory

Long-term notes this agent keeps between runs. One bullet per memory:

- [fact] The demo script lives in workspace/demo/hello.js. _(2026-10-28)_
- [gotcha] The sandbox refuses writes outside workspace/. _(2026-10-28)_
- [pref] The user wants short final answers. _(2026-10-29)_
```

tag — fact · pref · fix · gotcha. Anything else becomes fact.

text — one self-contained sentence.

date — when it was learned. Optional for hand-added lines.

> **On screen:** The file is a heading, a sentence of explanation, and bullets. Each bullet carries a tag, a self-contained text, and a date. That is the least clever format that still parses, and a human can edit it by hand.

03 · Parsing

Anything that is not a bullet is ignored.

SRC/MEMORY.JS · *source*

```
const m = line.match(/^\s*[-*]\s*\[([a-z]+)\]\s*(.+?)\s*(?:_\((\d{4}-\d{2}-\d{2})\)_)\s*$/i);
if (!m) continue;

const text = normalise(m[2]);
if (!text) continue;
entries.push({ tag: m[1].toLowerCase(), text, date: m[3] ?? 'undated' });
```

> Prose, headings and comments are safe in this file. Only a bullet with a tag becomes a memory.
> **On screen:** parseMemory scans line by line and keeps only the bullet shape. Whitespace is collapsed, so two copies of the same sentence become one memory. The date is optional. A hand-added line without one still parses as undated.

04 · Two access paths

Eager digest. Lazy recall.

Eager: digest() — The newest few entries, capped by count and characters, appended before step 1.

Lazy: recall — Search the whole file when a run needs the older detail that did not fit.

> The digest gives the agent a sense of continuity. Recall keeps the rest reachable without paying for it on every request.
> **On screen:** The first access path is eager: a small digest goes into the system prompt. The second is lazy: the recall tool searches the whole file when it is needed. One path makes the agent feel like it remembers. The other keeps the context window bounded.

05 · Writing

Remember, but stay bounded.

SRC/MEMORY.JS · *source*

```
const entry = {
  tag: MEMORY_TAGS.includes(tag) ? tag : 'fact',
  text: text.slice(0, this.options.maxEntryChars),
  date: today(this.now()),
};

const existing = entries.findIndex((e) => e.text.toLowerCase() === key);
if (existing !== -1) {
  entries[existing] = { ...entries[existing], date: entry.date, tag: entry.tag };
  // ...refresh, do not duplicate...
}

entries.push(entry);
const dropped = Math.max(0, entries.length - this.options.maxEntries);
if (dropped) entries.splice(0, dropped);
```

Dedupe — Same text refreshes the date instead of appending.

Cap each entry — 300 chars by default.

Drop oldest — keep the newest maxEntries.

> **On screen:** remember normalises the text, picks a tag, and caps the entry length. A duplicate refreshes the date instead of adding the same sentence twice. The file keeps only the newest entries. Unbounded memory becomes a context leak.

06 · The digest

Bounded twice. Honest about the tail.

SRC/MEMORY.JS · *source*

```
const newest = all.slice(-Math.max(1, entries)).reverse();
const lines = [];
let used = 0;
for (const entry of newest) {
  const line = formatEntry(entry);
  if (used + line.length + 1 > chars) break;
  lines.push(line);
  used += line.length + 1;
}

const hidden = all.length - lines.length;
const tail = hidden > 0
  ? '\n(' + hidden + ' older memories not shown - search with recall)'
  : '';
```

WHAT THE PROMPT GETS · *text*

```
## What you remember from earlier runs
- [fact] npm test needs Node 20. _(2026-10-20)_
- [gotcha] .env must never be edited. _(2026-10-22)_
(7 older memories not shown - search src/memory.md with the recall tool)
```

> **On screen:** The digest takes the newest entries first, then stops at the character budget. It counts what it left out and says so in the block. The model knows recall exists instead of assuming the older memories are gone.

07 · Searching

Newest first. Every word must match.

SRC/MEMORY.JS · *source*

```
search(query, { limit = 20 } = {}) {
  const words = normalise(query).toLowerCase().split(' ').filter(Boolean);
  const all = this.entries().reverse();
  const hits = words.length
    ? all.filter((e) => words.every((w) => e.text.toLowerCase().includes(w) || e.tag === w))
    : all;
  return hits.slice(0, Math.max(1, Number(limit) || 20));
}

forget(match) {
  const kept = entries.filter((e) => !e.text.toLowerCase().includes(needle));
  // ...save only if something was removed...
}
```

“node version” is narrower than “node”. Search is intentionally simple and explainable.

On screen: Search reverses the file so the newest memories come first. Every word in the query must appear in the text or match the tag. Forget removes every entry containing a phrase. Both paths rewrite the same plain file.

08 · The honest limit

No cache. No hidden state.

- 📄 The file is the store every method reads it fresh.
- ✍️ Hand edits count the next call sees them immediately.
- 🗒️ Diffable a memory change is a normal markdown diff.
- 🚫 Not a database no index, no query language, no background sync.

> **remember** The smallest format that survives a process is often a file you can read.
> **On screen:** There is no cache and no index. The file is the truth. If you edit or delete a line by hand, the next call sees it. That is a feature: memory you can diff and review like any other file.

09 · Recap

A small file with two jobs.

• 1 markdown file

• 2 access paths

• 4 tags

→

The file is the store. No cache, no index, no drift.

→

Digest eagerly. A bounded slice before step 1.

→

Recall lazily. Search the whole file on demand.

→

Bound the writes. Dedupe, cap, drop the oldest.

> **remember** Memory is not a database feature. It is a file the harness reads and the agent can add to.
> **On screen:** One markdown file, two access paths, four tags. Read it fresh, digest a bounded slice, recall the rest, and bound every write. That is how the agent remembers without losing control of the context window.